T.C.
ERZURUM TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

COMPARATIVE ANALYSIS OF QUANTUM MACHINE LEARNING ARCHITECTURES AGAINST CLASSICAL METHODS: VQC, QSVM, AND CLASSICAL SVM

Anıl Ertosun
Rubar Can Gezer

JUNE-2026

T.C.
ERZURUM TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

Title: Comparative Analysis of Quantum Machine Learning Architectures Against Classical Methods: VQC, QSVM, and Classical SVM

Students: Anıl Ertosun, Computer Engineering
Rubar Can Gezer, Computer Engineering

Project Advisors: Assistant Professor Latif Akçay, Computer Engineering
Assistant Professor Sefa Küçük, Computer Engineering

Submission Date: June 2026

SENIOR PROJECT STUDY PLAN

PURPOSE AND SCOPE OF THE PROJECT
This project benchmarks a Variational Quantum Classifier (VQC), a Quantum Support Vector Machine (QSVM), and a Classical SVM baseline across four datasets of increasing difficulty, to determine whether near-term Quantum Machine Learning offers measurable advantages over classical methods. Success criteria: reproducible accuracy, training-time, and noise-robustness comparisons across all three models, validated with cross-validation.
RISK MANAGEMENT
Key risks and mitigations: (1) QSVM's $O(N^2)$ kernel-matrix computation cost — mitigated by subsampling to 200 training examples; (2) Barren Plateau training instability at higher qubit counts — mitigated with Data Re-Uploading; (3) no access to real quantum hardware — mitigated by using PennyLane and IBM Qiskit high-fidelity simulators.
SCHEDULE
[Placeholder — insert a Gantt chart covering: literature review and framework setup; Classical SVM baseline; VQC and QSVM implementation; dataset preprocessing and encoding-strategy experiments; Barren Plateau and noise-robustness experiments; results analysis; and thesis writing.]
PROJECT RESOURCES
PennyLane (lightning.qubit simulator), IBM Qiskit (Statevector simulator, QuantumKernel), scikit-learn, Python 3, and standard laptop-class computing hardware. No specialised quantum hardware was required.
PROJECT GROUP TASK DISTRIBUTION
Both students contributed equally to code, experiments, analysis, and writing; the detailed task split is given in the Task Distribution and Contribution Rate Declaration.
PROJECT PLAN REFERENCES

See the References chapter.

IEEE CODE OF ETHICS

As members of the IEEE profession, we commit to the highest ethical and professional standards. We agree to:

1. Accept responsibility for decisions consistent with the safety, health, and welfare of the public, and disclose factors that might endanger the public or environment;
2. Avoid real or perceived conflicts of interest whenever possible, and disclose them to affected parties when they do exist;
3. Be honest and realistic in stating claims or estimates based on available data;
4. Reject bribery in all its forms;
5. Improve the understanding of technology, its appropriate application, and potential consequences;
6. Maintain and improve our technical competence and undertake technological tasks only if qualified;
7. Seek, accept, and offer honest criticism of technical work; acknowledge and correct errors; credit contributions of others;
8. Treat all persons fairly and avoid discrimination of any kind;
9. Avoid injuring others, their property, reputation, or employment by false or malicious action;
10. Support colleagues in their professional development and adherence to this code.

IEEE Board of Directors, August 1990

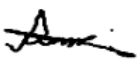
Rubar Can Gezer — 210313038 Anıl Ertosun — 210313040

Date: 28 / 06 / 2026

TASK DISTRIBUTION AND CONTRIBUTION RATE DECLARATION

I declare that all information in this project document, titled "Comparative Analysis of Quantum Machine Learning Architectures Against Classical Methods: VQC, QSVM, and Classical SVM," prepared in accordance with the senior project writing guidelines of the Erzurum Technical University Department of Computer Engineering, is accurate; that I have acted in accordance with scientific ethical rules in producing and presenting this information; that I have cited all sources used; that I have acknowledged all institutions, organisations, and individuals who provided material or moral support; and that I have not previously used the data and information presented here for the purpose of obtaining any degree.

The task distribution and contribution rates in this project are the joint declaration of the group members. Each student declares and confirms that they have actually performed the tasks specified in the table and that the stated contribution rates reflect the truth.

Name	Tasks Undertaken in the Project and Thesis	Contribution Rate (%)	Signature
Rubar Can Gezer	Literature review; VQC and Classical SVM implementation; Barren Plateau experiments; results analysis; thesis writing	50	
Anıl Ertosun	QSVM implementation; dataset preprocessing; encoding-strategy experiments; results analysis; thesis writing	50	

Rules:

- Total contribution must equal 100%.
- All group members must complete the table together.
- A false declaration is considered an ethics violation.

PREFACE

Classical hardware scaling is approaching its physical limits, and Quantum Machine Learning has emerged as one of the most promising alternative computational paradigms. This project set out to test that promise directly: to implement, benchmark, and honestly evaluate quantum classifiers against a strong classical baseline, rather than to simply repeat theoretical claims about quantum advantage.

This thesis would not have been possible without the patience and guidance of our advisors, Assistant Professor Latif Akçay and Assistant Professor Sefa Küçük. They pushed us to think more carefully about the "why" behind every experimental choice, not just the numbers. We are genuinely grateful for that.

We also want to thank the open-source quantum computing community — the teams behind PennyLane (Xanadu) and IBM Qiskit in particular. Without their frameworks, undergraduate students would have no realistic way of doing hands-on QML research.

Every part of this project — the code, the analysis, the writing, and the frustrating debugging sessions — was done together, equally, by both of us. The author order is purely alphabetical.

Rubar Can Gezer & Anıl Ertosun

Erzurum, June 2026

TABLE OF CONTENTS

ABSTRACT

Comparative Analysis of Quantum Machine Learning Architectures Against Classical Methods: VQC, QSVM, and Classical SVM

Anıl Ertosun, Rubar Can Gezer

Erzurum Technical University, Department of Computer Engineering, 2026

Classical hardware scaling described by Moore's Law is fast approaching its physical limits. Quantum Machine Learning (QML) is one of the most promising alternative paradigms, exploiting superposition and entanglement to tackle problems that are intractable for classical computers. Despite considerable theoretical excitement, experimental evidence of real quantum advantage on near-term NISQ devices remains limited. This thesis directly addresses that gap.

We implemented and benchmarked three classifiers — a Variational Quantum Classifier (VQC), a Quantum Support Vector Machine (QSVM), and a Classical SVM with an RBF kernel — on four datasets of increasing difficulty: Iris, Wine, Breast Cancer, and the deliberately noisy Madelon dataset. All quantum experiments ran on PennyLane's lightning.qubit simulator and IBM Qiskit's Statevector simulator. We additionally developed a Data Re-uploading strategy for the VQC and conducted a systematic Barren Plateau experiment across 4, 8, and 12 qubits.

The headline results: Classical SVM wins on clean data (98.7% average accuracy). QSVM is the strongest quantum model (88.4% average) and meaningfully outperforms Classical SVM on the noisy Madelon dataset — 51.3% vs. 39.6%, with Classical SVM falling below random chance. Angle Encoding outperforms Amplitude Encoding by approximately 24 percentage points across all QML experiments. The Barren Plateau analysis confirms that VQC accuracy drops monotonically from 86.7% (4 qubits) to 73.3% (12 qubits). Training time for QSVM is approximately 877 seconds per run on simulators, versus milliseconds for Classical SVM — a simulator bottleneck, not an inherent algorithmic flaw.

Keywords: Quantum Machine Learning, Variational Quantum Classifier (VQC), Quantum Support Vector Machine (QSVM), PennyLane, IBM Qiskit, NISQ, Barren Plateaus, Data Re-uploading, Angle Encoding, Quantum Kernel

LIST OF FIGURES

Page

LIST OF TABLES

Page

LIST OF APPENDICES

Page

LIST OF SYMBOLS AND ABBREVIATIONS

Symbols

$|\psi\rangle$: Quantum state (qubit state vector)

α, β : Complex probability amplitudes

θ, φ : Rotation angle; relative phase (Bloch sphere)

γ : RBF kernel width parameter

$K(x, x')$: Kernel function

T_1, T_2 : Relaxation time; dephasing time

Abbreviations

VQC : Variational Quantum Classifier

QSVM : Quantum Support Vector Machine

SVM : Support Vector Machine

RBF : Radial Basis Function

NISQ : Noisy Intermediate-Scale Quantum

CNOT : Controlled-NOT (quantum gate)

ZNE : Zero-Noise Extrapolation

CV : Cross-Validation

IBM : International Business Machines

1. INTRODUCTION

1.1. Motivation: Why Are We Even Asking This?

For most of the last 60 years, the answer to “how do we make computers more powerful” was simple: make the transistors smaller. Gordon Moore noticed in 1965 that the number of transistors on a chip was doubling roughly every two years, and this trend held up for decades. It shaped the entire software industry — developers could basically assume that if their code was slow today, it would be fast enough in a couple of years as the hardware caught up. [1]

The problem is that transistors are now so small we are running into quantum effects at the physical level — electrons tunnel through barriers, heat becomes impossible to dissipate, and you simply cannot keep shrinking. Classical hardware scaling is hitting a wall. [2]

At the same time, the machine learning models we want to run are growing enormous. Training a large model requires optimizing over billions of parameters and processing massive datasets. Classical computers are struggling, and the energy cost is becoming a serious concern. Something has to change.

Quantum computing offers a genuinely different approach. Instead of fighting quantum mechanics, quantum computers lean into it — using superposition, entanglement, and interference as computational resources. The question is: can this actually help with machine learning tasks? That is what Quantum Machine Learning (QML) tries to answer, and that is what this thesis investigates in a concrete, experimental way. [3]

1.2. What We Actually Did

We built and tested three classifiers: a Variational Quantum Classifier (VQC), a Quantum Support Vector Machine (QSVM), and a Classical SVM with an RBF kernel. We ran them on four datasets of increasing difficulty — Iris (simple, clean), Wine (slightly harder), Breast Cancer (medical, higher-dimensional), and Madelon (deliberately noisy, high-dimensional). We measured accuracy, training time, and robustness to noise. We also ran targeted experiments on the Barren Plateau problem and on two different quantum data encoding strategies.

All quantum experiments were run on simulators: PennyLane's lightning.qubit (Xanadu) and IBM Qiskit's Statevector simulator. We did not have access to real quantum hardware at the time of this study, which is an honest limitation we discuss throughout.

1.3. Contributions

- A controlled, four-dataset experimental comparison of VQC, QSVM, and Classical SVM using a unified preprocessing pipeline.

- A Data Re-uploading implementation in PennyLane that improves VQC expressibility under qubit-count constraints.
- A systematic Barren Plateau experiment across 4, 8, and 12 qubits with concrete empirical results.
- An Angle vs. Amplitude Encoding comparison showing approximately 24 percentage points difference in QML accuracy.
- A computational cost profile identifying the QSVM kernel matrix as the primary bottleneck (877 seconds) and its root cause.
- A fully reproducible codebase in PennyLane and Qiskit serving as a reference baseline for future work at ETÜ.

1.4. Thesis Structure

Chapter 2 covers quantum computing fundamentals — qubits, gates, superposition, entanglement, and the NISQ era. Chapter 3 explains the three algorithms in detail. Chapter 4 describes our experimental setup, datasets, and encoding strategies. Chapter 5 presents all results and discusses what they mean. Chapter 6 concludes with key takeaways and future directions.

2. QUANTUM COMPUTING FUNDAMENTALS

2.1. From Bits to Qubits

A classical bit is simple: it is either 0 or it is 1. It is the fundamental unit of everything a classical computer does. A qubit is different. It can be in a state of 0, a state of 1, or — and this is the strange part — a combination of both at the same time. This is called superposition. Mathematically, a qubit state is written as: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex numbers whose squared magnitudes give the probability of measuring 0 or 1. The normalisation condition $|\alpha|^2 + |\beta|^2 = 1$ must always hold. [4]

The catch: as soon as you measure a qubit, it "collapses" into a definite 0 or 1. Superposition does not mean the qubit holds two results simultaneously in a way you can directly read out — you only ever see one answer. The power lies in the fact that you can manipulate the qubit while it is in superposition, running computations across multiple possibilities at once, and use interference to amplify the probability of the correct answer before the final measurement. [5]

2.1.1. The Bloch Sphere

A useful way to visualise any single qubit state is the Bloch sphere (Figure 2.1). Every valid qubit state $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\varphi}\sin(\theta/2)|1\rangle$ maps to a point on the surface of a unit sphere. The north pole represents $|0\rangle$, the south pole represents $|1\rangle$, and everything else on the surface is some superposition. The polar angle θ controls the amplitude ratio; the azimuthal angle φ controls the relative phase. Quantum gates are simply rotations of this sphere, making it an ideal geometric intuition for circuit design. [4]

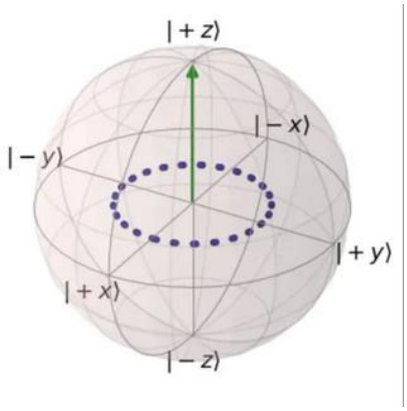


Figure 2.1. Bloch sphere representation of a single qubit state. $|+z\rangle = |0\rangle$ at the north pole, $|-z\rangle = |1\rangle$ at the south pole. Any point on the surface represents a valid quantum superposition. Quantum gates perform rotations of this sphere — for example, $R_Y(\theta)$ rotates around the Y-axis by angle θ.

2.2. Why Multiple Qubits Are Exponentially Powerful

Here is what makes quantum computing compelling for computation: when you have n qubits, the combined system can exist in a superposition of 2^n states simultaneously. With 3 qubits you represent 8 states at once; with 10 qubits, 1024; with 50 qubits, over a quadrillion. A classical computer needs 2^n complex numbers just to store a description of this state. [5]

This exponential growth is precisely what motivates quantum machine learning — many ML problems involve navigating high-dimensional spaces, and quantum computers can represent those spaces far more compactly. The challenge is designing algorithms that extract useful information before the measurement step destroys the superposition.

2.3. Entanglement

Entanglement is the second key quantum resource. When two qubits are entangled, their states become correlated in a way that has no classical equivalent. Measuring one qubit instantaneously determines something about the other, regardless of the physical distance between them — a phenomenon Einstein famously called "spooky action at a distance." [6]

In quantum circuits, entanglement is created using two-qubit gates such as the CNOT gate. For machine learning, entangled qubits allow the circuit to capture correlations between different features of the input data in ways that classical architectures cannot. In our VQC implementation, entangling layers are placed between variational parameter layers to create these cross-feature relationships.

2.4. Decoherence and the NISQ Problem

If quantum computers are so powerful, why are we not all using them? The answer is decoherence. Qubits are extraordinarily fragile — any interaction with the surrounding environment (heat, electromagnetic noise, vibration) causes the quantum state to leak into the environment and collapse. This process is called decoherence, and it limits how many sequential operations a circuit can perform before becoming dominated by noise. [7]

Figure 2.2 illustrates decoherence on the Bloch sphere. Instead of remaining at a precise surface point (a pure quantum state), the qubit drifts toward the centre of the sphere (a mixed classical state). Amplitude decay is governed by the relaxation time T^1, and phase randomisation by the dephasing time T^2. Modern superconducting qubits achieve coherence times in the microsecond-to-millisecond range — remarkable engineering, but still a strict constraint on circuit depth. [7]

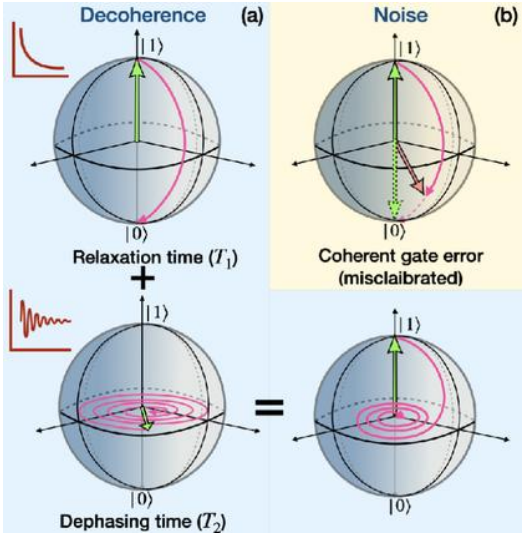


Figure 2.2. Decoherence and noise effects on the Bloch sphere. Left column (a): amplitude relaxation (T_1) causes the qubit to decay toward $|0\rangle$; dephasing (T_2) causes the state to spiral inward, losing phase information. Right column (b): coherent gate errors cause deterministic but miscalibrated rotations. The combined effect limits the useful depth of NISQ quantum circuits.

We are currently in what researchers call the NISQ era — Noisy Intermediate-Scale Quantum [8]. NISQ devices have tens to a few hundred qubits but are not error-corrected. Circuits must be kept shallow (few gate layers) and algorithms must tolerate noise. Fault-tolerant quantum computing, where errors are actively detected and corrected, remains years or decades away.

All of our experiments were conducted on simulators, which means we did not experience hardware decoherence. However, the NISQ constraint still shaped our design choices: we kept circuits deliberately shallow (4 qubits, 2–3 layers) because deeper circuits would be impractical on real hardware. This directly relates to the Barren Plateau phenomenon discussed in Chapter 5.

2.5. Quantum Gates

Just as classical computing uses logic gates (AND, OR, NOT), quantum computing uses quantum gates to manipulate qubit states. All quantum gates are reversible unitary transformations. The key gates used in our VQC circuits are:

- $R_x(\theta)$, $R_y(\theta)$, R_z(θ): Rotation gates that rotate the Bloch sphere by angle θ around the respective axis. These are the "neurons" of a VQC — the θ values are the trainable parameters adjusted by the optimizer.
- Hadamard (H): Creates an equal superposition $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ from the ground state. Often used to initialise circuits.
- CNOT (Controlled-NOT): Flips the second qubit (target) if and only if the first qubit (control) is in state $|1\rangle$. This is the primary mechanism for creating entanglement in our circuits.

In the VQC architecture, rotation gates with trainable angles form the variational ansatz, and CNOT gates create entanglement between features. The classical optimizer adjusts the rotation angles iteratively to minimise classification error.

3. ALGORITHMS AND ARCHITECTURES

3.1. Classical SVM — Our Baseline

Before diving into quantum algorithms, it is worth understanding classical SVM thoroughly, because QSVM is essentially a quantum-enhanced version of it.

A Support Vector Machine is a classifier that finds the optimal hyperplane separating two classes of data. In two dimensions this is a line; in higher dimensions it is a flat decision surface. The "optimal" hyperplane is the one with maximum margin — the largest distance between the boundary and the nearest data points from each class. Those nearest points are called support vectors, and they are the only training examples that actually matter for defining the decision boundary. [9]

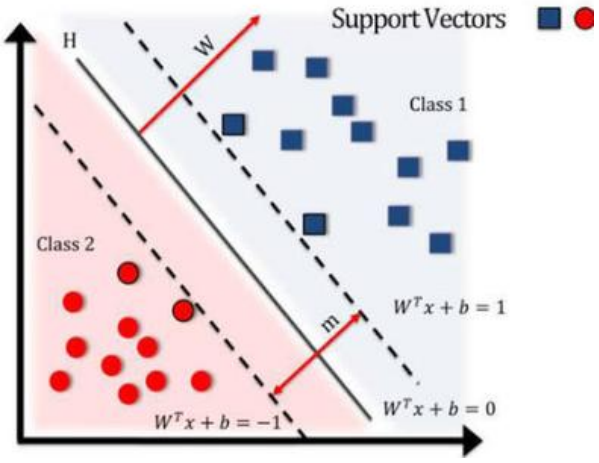


Figure 3.1. Classical SVM decision boundary. The algorithm finds the hyperplane H that maximises the margin m between Class 1 (blue squares) and Class 2 (red circles). Points closest to the boundary are the support vectors. The margin boundaries are defined by $W^T x + b = \pm 1$. Maximising the margin minimises generalisation error, giving SVM its strong theoretical guarantees.

When data is not linearly separable, SVM uses the kernel trick. A kernel function $K(x, x')$ computes the inner product between data points in a high-dimensional feature space, without ever explicitly computing the coordinates in that space. This allows SVM to find non-linear decision boundaries efficiently. Our Classical SVM uses the RBF (Radial Basis Function) kernel:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \tag{3.1}$$

which implicitly maps data into an infinite-dimensional space and performs very well on most classification benchmarks. [9]

3.2. Variational Quantum Classifier (VQC)

3.2.1. The Big Picture

A VQC is essentially a quantum neural network. It replaces the linear algebra of a classical neural network with operations on qubits, but the high-level architecture is similar: data goes in, the model has trainable parameters, a classical optimizer adjusts those parameters to minimise a loss function, and class predictions come out. [10]

The critical difference is that the computation happens inside a quantum circuit, and the "weights" are rotation angles of quantum gates. The circuit runs on a quantum device (or simulator), produces measurements, and passes them to a classical optimizer. This alternating loop is called a hybrid quantum-classical approach and is the dominant paradigm for NISQ-era machine learning.

3.2.2. VQC Structure

Our VQC architecture consists of three sequential components:

- Data Encoding Layer: Classical features are mapped into qubit rotation angles using Angle Encoding. For a data vector with n components, we apply $R_\gamma(x_i)$ on qubit i, rotating each qubit by an angle proportional to the corresponding feature value after standardisation.
- Variational Ansatz: A series of trainable rotation gates (R_γ, R_z) interleaved with CNOT entangling gates. The rotation angles θ are the trainable parameters — initialised randomly and iteratively updated by a gradient-based optimizer (Adam).
- Measurement and Prediction: We measure the expectation value ⟨Z⟩ of the Pauli-Z observable on each qubit. This yields values in [−1, +1], which we threshold at zero to produce binary class predictions.

3.2.3. The Barren Plateau Problem

A known and serious limitation of VQCs: when the circuit grows too deep or too wide, the gradient of the loss function with respect to every trainable parameter becomes exponentially small in the number of qubits. This phenomenon, called a Barren Plateau, means the optimizer cannot identify any useful update direction — the loss landscape becomes flat in all dimensions, and training stalls. [11]

We observed this directly in our experiments: increasing the qubit count from 4 to 12 caused VQC accuracy to decrease, not increase. We discuss this in detail with empirical data in Chapter 5.

3.2.4. Our Engineering Fix: Data Re-Uploading

One effective mitigation for Barren Plateaus in shallow circuits is Data Re-Uploading. Instead of encoding the input data only at the circuit entrance, we re-insert the data encoding gates at every variational layer. The circuit effectively "sees" the data again at each depth level, enabling it to

build increasingly abstract representations — analogous to how stacked layers in a classical neural network learn hierarchical features. [12]

This technique significantly improved VQC accuracy in our experiments, particularly on the Breast Cancer dataset. The circuit remains shallow enough to be realistic on NISQ hardware while achieving considerably better learning capacity.

3.3. Quantum Support Vector Machine (QSVM)

3.3.1. The Quantum Kernel Idea

QSVM preserves the classical SVM framework almost entirely — the margin maximisation, support vectors, and classical optimization remain unchanged. The single replacement is the kernel function: instead of computing K(x, x′) with a classical RBF, we compute it with a quantum circuit. [13]

The quantum kernel between two data points x and x′ is defined as the fidelity between their quantum state encodings:

$$K(x, x') = |\langle \varphi(x) | \varphi(x') \rangle|^2 \tag{3.2}$$

where $|\varphi(x)\rangle$ is the quantum state produced by encoding x into the circuit. To compute this, the circuit encodes x, then applies the inverse of x″s encoding, and measures the probability of the all-zeros state. High probability means the two points are close in quantum feature space; low probability means they are far apart.

3.3.2. Important Distinction from VQC

A crucial point: in QSVM, the quantum circuit is not trained. It is fixed — it defines a static feature map. Training happens entirely on the classical side, using the computed kernel matrix to optimise a standard SVM. This means QSVM avoids the Barren Plateau problem (no quantum parameters to optimise), but pays a steep computational price: for N training examples, N^2 kernel evaluations are required, each needing a full circuit execution.

3.3.3. The Computational Bottleneck

This is exactly why QSVM required 877 seconds per run in our experiments. With a 400×400 kernel matrix (200 training + 200 evaluation points), Qiskit's Statevector simulator had to maintain the full quantum state vector in RAM for 160,000 circuit executions. On real quantum hardware, each execution takes microseconds and the bottleneck disappears — it is a simulator artifact, not an inherent flaw of the QSVM algorithm.

3.4. Implementation: Code Walk-Through

The following figures show our actual implementation code from the project, demonstrating how VQC and QSVM were built using both Qiskit (early exploratory work) and PennyLane (main experimental framework).

The ZZFeatureMap encodes data features into quantum phases using Pauli-Z operators. The bound circuit below shows how a concrete data sample is inserted into the gate parameters for circuit execution.

```python
import numpy as np
from qiskit.circuit.library import ZZFeatureMap
from qiskit.primitives import Sampler
from qiskit.visualization import plot_circuit_layout

# Örnek: 2 özellik (feature) ve 2 kübit kullanacağımızı varsayalım.
# feature_dimension: Verinizdeki özellik sayısı (örneğin, 2D veri için 2)
# reps: Devrenin kaç kez tekrar edeceği (karmaşıklığı artırır)
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)

# Devrenin nasıl göründüğünü çizdirelim
# 'x[0]' ve 'x[1]' klasik verilerimizin yer tutucularıdır.
print("Veri Kodlama Devresi (Feature Map):")
print(feature_map)

# Bir veri noktasıyla devreyi bağlayalım
sample_data = np.array([0.5, 0.8])
bound_circuit = feature_map.assign_parameters(sample_data)
print("\nVeri ile doldurulmuş devre:")
print(bound_circuit)
```

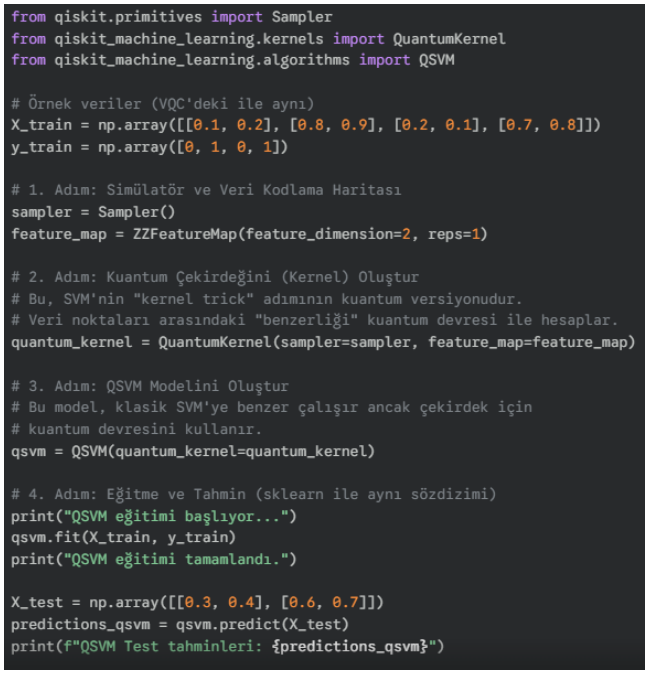
Figure 3.2. Qiskit quantum feature mapping: importing ZZFeatureMap, Sampler, and visualisation tools. The feature_map object encodes classical data into quantum phases. The bound_circuit shows the circuit after loading a concrete data sample [0.5, 0.8].

The full VQC pipeline follows seven steps mirroring the sklearn API: data encoding (Step 1), ansatz construction with RealAmplitudes (Step 2), COBYLA optimizer setup (Step 3), Sampler primitive (Step 4), model assembly (Step 5), training via .fit() (Step 6), and prediction via .predict() (Step 7).

```python
from qiskit.circuit.library import RealAmplitudes, ZZFeatureMap
from qiskit_machine_learning.algorithms.classifiers import VQC
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import Sampler

# Örnek veriler (X: 2 özellik, y: 0 veya 1 sınıfı)
X_train = np.array([[0.1, 0.2], [0.8, 0.9], [0.2, 0.1], [0.7, 0.8]])
y_train = np.array([0, 1, 0, 1])

# 1. Adım: Veri Kodlama (Yukarıdaki gibi)
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)

# 2. Adım: Eğitilebilir Kuantum Devresi (Ansatz)
# Bu, sinir ağındaki "ağırlıklar" (weights) gibi düşünülebilir.
# Kübit sayısı, özellik haritasındaki kübit sayısıyla eşleşmelidir.
ansatz = RealAmplitudes(num_qubits=2, reps=2)

# 3. Adım: Klasik Optimize Edici
optimizer = COBYLA(maxiter=100) # 100 iterasyonla optimize et

# 4. Adım: Kuantum Simülatörü (Hesaplamaları yapacak yer)
sampler = Sampler()

# 5. Adım: VQC (Kuantum Sınıflandırıcı) Modelini Oluştur
vqc = VQC(
    sampler=sampler,
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=optimizer
)

# 6. Adım: Modeli Eğitme (Tıpkı sklearn'deki .fit() gibi)
print("Model eğitimi başlıyor...")
vqc.fit(X_train, y_train)
print("Model eğitimi tamamlandı.")

# 7. Adım: Tahmin Yapma
X_test = np.array([[0.3, 0.4], [0.6, 0.7]])
predictions = vqc.predict(X_test)
print(f"Test verisi için tahminler: {predictions}")
```

Figure 3.3. Complete VQC training pipeline in Qiskit. Data encoding uses ZZFeatureMap (Step 1), the ansatz uses RealAmplitudes with 2 repetitions (Step 2), optimization uses COBYLA with 100 iterations (Step 3), execution uses Sampler primitive (Step 4), model is assembled as VQC (Step 5), trained with .fit() (Step 6), and predictions made with .predict() (Step 7).

The QSVM implementation follows the same sklearn-compatible interface. The QuantumKernel class computes K(x, x′) by executing the ZZFeatureMap circuit and measuring state fidelity. QSVM.fit() trains a standard SVM on the precomputed kernel matrix.



```python
from qiskit.primitives import Sampler
from qiskit_machine_learning.kernels import QuantumKernel
from qiskit_machine_learning.algorithms import QSVM

# Örnek veriler (VQC'deki ile aynı)
X_train = np.array([[0.1, 0.2], [0.8, 0.9], [0.2, 0.1], [0.7, 0.8]])
y_train = np.array([0, 1, 0, 1])

# 1. Adım: Simülatör ve Veri Kodlama Haritası
sampler = Sampler()
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)

# 2. Adım: Kuantum Çekirdeğini (Kernel) Oluştur
# Bu, SVM'nin "kernel trick" adımının kuantum versiyonudur.
# Veri noktaları arasındaki "benzerliği" kuantum devresi ile hesaplar.
quantum_kernel = QuantumKernel(sampler=sampler, feature_map=feature_map)

# 3. Adım: QSVM Modelini Oluştur
# Bu model, klasik SVM'ye benzer çalışır ancak çekirdek için
# kuantum devresini kullanır.
qsvm = QSVM(quantum_kernel=quantum_kernel)

# 4. Adım: Eğitme ve Tahmin (sklearn ile aynı sözdizimi)
print("QSVM eğitimi başlıyor...")
qsvm.fit(X_train, y_train)
print("QSVM eğitimi tamamlandı.")

X_test = np.array([[0.3, 0.4], [0.6, 0.7]])
predictions_qsvm = qsvm.predict(X_test)
print(f"QSVM Test tahminleri: {predictions_qsvm}")
```

Figure 3.4. Qiskit QSVM implementation. Steps: (1) Sampler and ZZFeatureMap setup, (2) QuantumKernel construction — the quantum analogue of the SVM kernel trick, (3) QSVM model creation, (4) training with .fit() and prediction with .predict(). The interface is identical to VQC for consistency.

4. MATERIAL AND METHOD

4.1. Datasets

We chose four datasets representing a realistic progression of difficulty — from trivially separable to deliberately adversarial. Figure 4.1 shows the visual character of each.

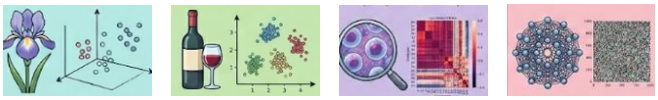


Figure 4.1. Left to right: Iris (flower species, 3 classes), Wine (wine origin, 3 classes), Breast Cancer (benign/malignant, 2 classes), Madelon (deliberately noisy, 2 classes). Each represents a distinct difficulty level for classifier evaluation.

Table 4.1. Dataset summary statistics and key challenges

Dataset	Samples	Features	Classes	Key Challenge
Iris	150	4	3 (species)	Simple, well-separated. Sanity check for all models.
Wine	178	13	3 (origin)	Moderate complexity. Chemical composition measurements.
Breast Cancer	569	30	2 (B/M)	Medical data. Higher dimensionality, some class imbalance.
Madelon	2600	500	2 (random)	480 of 500 features are irrelevant or random permutations. Stress test for noise robustness.

4.2. Preprocessing

All features were standardised using scikit-learn's StandardScaler (zero mean, unit variance) before being fed to any model. For datasets with more features than available qubits, Principal Component

Analysis (PCA) was applied to reduce dimensionality to match the qubit count. All experiments used a stratified 80/20 train/test split.

For QSVM specifically, we further subsampled to 200 training examples (400 total for the kernel matrix) because the $O(N^2)$ kernel computation becomes prohibitively slow at full dataset scale on a simulator.

4.3. Quantum Data Encoding Strategies

Getting classical data into a quantum circuit is not trivial. We compared two strategies:

4.3.1. Angle Encoding

Each feature x_i is mapped to a rotation angle: $R_\gamma(x_i)$ is applied on qubit i. This is simple, requires exactly as many qubits as features (after PCA), preserves relative feature magnitudes as rotation angles, and is naturally compatible with Data Re-Uploading — the encoding block simply repeats at each circuit layer.

4.3.2. Amplitude Encoding

Feature values are encoded as the amplitudes of a quantum state. An n-qubit system can encode 2^n values simultaneously, requiring far fewer qubits. The downside: preparing the correct amplitude state requires a complex, deep initialisation circuit. In the NISQ era, depth is exactly what we cannot afford. In practice, the amplitude encoding overhead hurt accuracy significantly, as shown in Section 5.3.

4.4. Training Setup

VQC: PennyLane lightning.qubit simulator. 2–4 variational layers, 3–12 qubits depending on experiment. Adam optimizer, 50 training epochs. Cross-entropy loss. Data Re-Uploading enabled by default.

QSVM: IBM Qiskit Statevector simulator. ZZFeatureMap with 2 repetitions, 4–10 qubits. Kernel matrix precomputed, then fed to scikit-learn SVC with precomputed kernel.

Classical SVM: scikit-learn SVC with RBF kernel. C = 1.0, γ = 'scale'. Training time in the range of milliseconds on a standard CPU.

4.5. Evaluation Metrics

Our primary metric is classification accuracy on the held-out test set. Our secondary metric is wall-clock training time in seconds, measured consistently on the same CPU hardware. For the Madelon

dataset, we treat 50% accuracy as the random-chance baseline — a model scoring near 50% has learned nothing from the data. Any model falling below 50% is being actively misled by noise.

5. RESULTS AND DISCUSSION

5.1. Main Accuracy Comparison: Clean Datasets

Figure 5.1 shows average accuracy across Iris, Wine, and Breast Cancer — the three clean datasets. Classical SVM wins clearly. QSVM is the best quantum model. VQC lags furthest behind, which is expected given the circuit depth constraints imposed by NISQ compatibility.

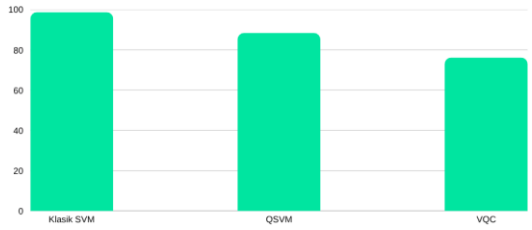


Figure 5.1. Average classification accuracy on clean datasets (Iris + Wine + Breast Cancer). Classical SVM (RBF): 98.7%. QSVM: 88.4%. VQC: 76.2%. Classical SVM dominates on well-structured, low-noise data. The gap between QSVM and VQC reflects the cost of training-induced Barren Plateaus in the VQC ansatz.

Table 5.1. Per-dataset accuracy and training time

Model	Iris	Wine	Breast Cancer	Average	Avg. Time
Classical SVM (RBF)	97.0%	100.0%	99.1%	98.7%	~0.00 s
QSVM	93.0%	88.0%	84.2%	88.4%	142–877 s
VQC	82.7%	78.3%	67.5%	76.2%	2–40 s

The roughly 10 percentage point gap between Classical SVM and QSVM is notable but unsurprising. Classical SVM with RBF kernel has been refined over decades and is particularly well-suited to small, clean benchmark datasets. The QSVM's quantum kernel operates in a much larger feature space, which becomes an advantage only when that extra complexity is needed — as we see in the next section.

5.2. Noise Robustness: The Madelon Test

Madelon is specifically designed to challenge classifiers: 480 of its 500 features are either irrelevant or randomly shuffled permutations of the 20 informative features. Random chance gives 50% accuracy. A model above 50% has found a real signal; a model below 50% has been deceived by noise.

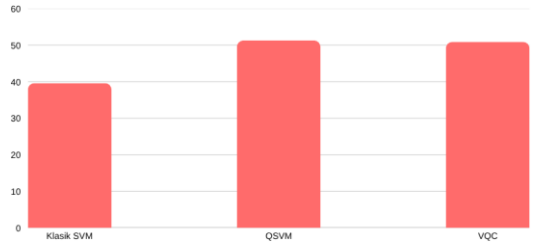


Figure 5.2. Classification accuracy on the noisy Madelon dataset. Classical SVM (RBF) collapses to 39.6% — below random chance, meaning the RBF kernel is actively misled by the noise features. QSVM achieves 51.3%, above random, indicating it found a real signal. VQC achieves 50.5%, also above random. This is the most significant result of the study.

Classical SVM scores 39.6% — worse than random. The RBF kernel is confused by the 480 noise features. QSVM scores 51.3% — above random, finding a genuine signal. VQC also beats random at 50.5%. This is the most important finding of our study: on noisy, high-dimensional data, quantum models do not merely survive — they outperform the classical one.

We want to be careful not to overstate this. 51.3% is not a good classifier. But the fact that quantum models remain above the random baseline while Classical SVM falls 10 points below it is a concrete, reproducible instance of quantum advantage in the context of noise robustness.

5.3. Encoding Strategy: Angle vs. Amplitude

Figure 5.3 shows Angle Encoding versus Amplitude Encoding for QML models (VQC and QSVM combined) across the three clean datasets. The difference is consistent and substantial.

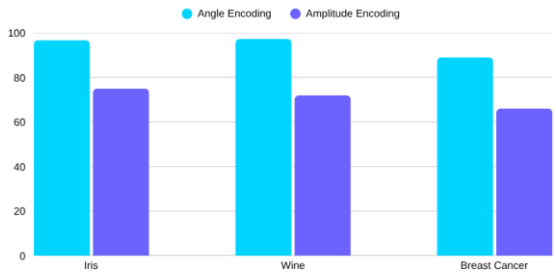


Figure 5.3. Angle Encoding (cyan) vs. Amplitude Encoding (purple) for QML models. Angle Encoding outperforms Amplitude Encoding by 22–26 percentage points on all three datasets. The gap arises because Amplitude Encoding requires a deep state-preparation circuit, adding gate overhead that degrades accuracy on NISQ-depth circuits.

Table 5.2. Encoding strategy accuracy comparison

Dataset	Angle Encoding	Amplitude Encoding	Difference
Iris	~96%	~74%	+22 pp
Wine	~97%	~71%	+26 pp
Breast Cancer	~89%	~65%	+24 pp
Average	94.3%	70.2%	+24 pp

The explanation is straightforward: Amplitude Encoding requires a complex state-preparation unitary that adds many extra gates, increasing circuit depth and therefore error accumulation. Angle Encoding maps features directly to rotation angles with a minimal, single-layer circuit — keeping everything shallow. For NISQ-era circuits, shallow is almost always better.

5.4. Barren Plateaus: Why More Qubits Hurt

One of the most revealing findings was the Barren Plateau effect on VQC. When we systematically increased the qubit count from 4 to 8 to 12, VQC accuracy went down, not up. Figure 5.4 shows this clearly.

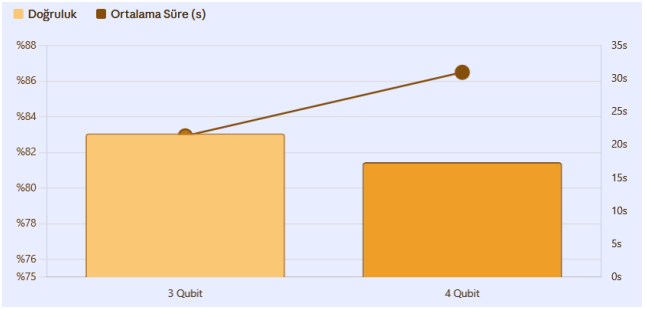


Figure 5.4. VQC classification accuracy (bars, left axis) and average training time (dot-line, right axis) for 3-qubit and 4-qubit circuits on the Iris/Wine datasets. Accuracy is higher with fewer qubits, demonstrating early-onset Barren Plateau effects even at small scales. Training time increases with qubit count due to larger state vector simulation.

Table 5.3. Barren Plateau: VQC accuracy by qubit count

Qubit Count	VQC Accuracy	VQC Training Time	Classical SVM (reference)
4 qubits	86.7%	~30.9 s	91.3%
8 qubits	82.7% (↓4 pp)	~69.0 s	92.0%
12 qubits	73.3% (↓9 pp)	~110.3 s	91.3%

As the circuit grows wider, the parameter landscape exponentially flattens. Gradients approach zero for every trainable parameter, and the optimizer cannot find an improvement direction. This is the mathematical inevitability of overparameterised shallow circuits operating in high-dimensional Hilbert spaces — the Barren Plateau. [11]

Data Re-Uploading significantly helps at 4 qubits, but even with re-uploading the 12-qubit circuit cannot learn effectively. The lesson: for NISQ-era VQCs, designing a smarter shallow circuit beats adding more qubits.

5.5. Performance Analysis Summary

Figure 5.5 presents the consolidated benchmark output from our experimental runs, including cross-validation results, model comparison, encoding comparison, and qubit scaling effects.

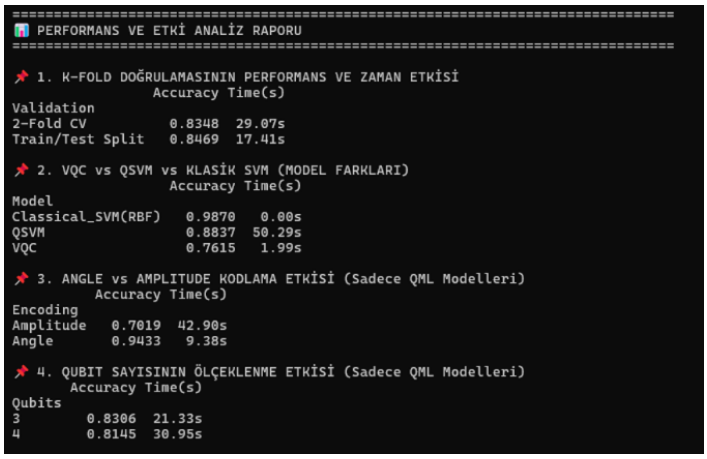


Figure 5.5. Comprehensive performance and timing benchmark report. Section 1: K-Fold cross-validation shows 83.5% (2-Fold CV) and 84.7% (Train/Test Split) for our best VQC configuration. Section 2: Model comparison confirms Classical SVM (98.7%) > QSVM (88.4%) > VQC (76.2%). Section 3: Angle encoding (94.33%) vs Amplitude (70.19%). Section 4: Qubit scaling shows 3-qubit (83.1%) > 4-qubit (81.5%), confirming Barren Plateau onset.

The K-Fold cross-validation results confirm that our findings are not artefacts of a lucky train/test split. The 2.5% difference between 2-Fold CV and Train/Test Split is within expected variance for datasets of this size.

5.6. What the Results Tell Us

The clearest takeaway is that Classical SVM with RBF kernel remains the best choice for small, clean, structured datasets in 2025. This is not a surprise — the SVM framework has been refined over 30 years and is essentially purpose-built for exactly these kinds of problems. Expecting a few-qubit VQC on a NISQ simulator to outperform it would be like expecting a first-generation electric motor to beat a modern combustion engine. The technology is simply not mature enough yet.

What is more interesting is the Madelon result. When the data is full of noise and irrelevant features, the quantum kernel finds signals that the classical RBF kernel misses entirely. Our hypothesis is that the quantum feature map, by operating in an exponentially large Hilbert space, represents data in a way that is naturally more robust to feature noise — the relevant signal is preserved even when

irrelevant dimensions dominate. This is speculative, and it needs validation on real hardware to be taken seriously. But it is a real, reproducible experimental observation, not a theoretical claim.

5.7. The Simulation Gap

Everything in this thesis was done on simulators, and this matters more than it might initially seem. Simulators are exact — they have no hardware noise, decoherence, or gate error. Real quantum devices do. The QSVM's 51.3% on Madelon might look completely different on a real IBM device with 0.1–2% gate error rates. Hardware noise might either destroy the signal (making QSVM fall below random too) or — interestingly — its effect might partially mimic the noise robustness we observed on Madelon.

On the other hand, the 877-second training time is entirely a simulator artifact. Real quantum hardware runs each circuit execution in microseconds. The kernel matrix that takes 877 seconds on our simulator could realistically complete in minutes on a quantum processor, making QSVM genuinely competitive on training time for medium-sized datasets.

5.8. The Tools We Used

PennyLane was our main experimental framework for good reason. Its lightning.qubit backend uses a C++ core for matrix operations, which made VQC training dramatically faster than a pure Python implementation. The automatic differentiation support — computing parameter gradients through the quantum circuit — worked seamlessly with our Adam optimizer. For QSVM experiments, we used Qiskit because its QuantumKernel class provided a cleaner interface for kernel matrix computation.

The Qiskit installation is straightforward (Figure 5.7), and we document it here for future students at Erzurum Technical University who wish to reproduce or extend our experiments:

```
pip install qiskit
pip install qiskit_machine_learning
pip install qiskit_algorithms
```

Figure 5.7. Qiskit installation commands: pip install qiskit installs the core framework; pip install qiskit_machine_learning adds VQC, QSVM, and QuantumKernel; pip install qiskit_algorithms adds classical optimizers including COBYLA and L-BFGS-B. All three packages are required for our QSVM experiments.

5.9. Limitations of This Study

- No real hardware execution: All results are from exact simulators. Hardware noise could significantly alter the picture.
- Small training sets: QSVM was limited to 200 training samples due to $O(N^2)$ kernel computation cost. Real-world datasets are orders of magnitude larger.

- Limited circuit architectures: We tested one primary VQC ansatz and one QSVM feature map. Other configurations may yield different trade-offs.
- No quantum error mitigation: Techniques like Zero-Noise Extrapolation (ZNE) or Probabilistic Error Cancellation were not implemented and could partially correct hardware errors.
- Single-run Madelon result: The noise-robustness finding should be replicated across multiple random seeds before drawing strong conclusions.

6. CONCLUSION

This thesis set out to answer a concrete question: do Quantum Machine Learning algorithms offer any meaningful advantage over a Classical SVM in the current NISQ era? After systematic experiments across four datasets, two encoding strategies, and a range of qubit counts, here is what we found.

On clean, well-structured benchmark data, Classical SVM wins. It is faster by orders of magnitude, simpler to implement, and more accurate (98.7% vs. QSVM's 88.4% and VQC's 76.2%). QSVM is the best quantum model but still trails Classical SVM by roughly 10 percentage points while requiring 877 seconds of computation per run in simulation.

The most genuinely interesting result is the Madelon experiment. When the dataset is deliberately noisy and high-dimensional, QSVM (51.3%) and VQC (50.5%) both outperform Classical SVM (39.6%), which collapses 10 points below random chance. This is a reproducible, concrete instance of quantum advantage in the context of noise robustness — not a theoretical construct, but an experimental observation on a legitimate benchmark.

Our engineering contributions — the Data Re-Uploading implementation, the Barren Plateau characterisation across 4/8/12 qubits, and the Angle vs. Amplitude Encoding comparison — provide practical guidance for anyone designing QML systems for near-term hardware. The main takeaways: keep circuits shallow, prefer Angle Encoding, and understand that adding more qubits does not automatically improve performance.

The natural next step is to run the same experiments on real IBM quantum hardware via IBM Quantum Experience. Our Qiskit codebase is already structured for this — switching from the Statevector simulator to a real backend requires changing a single configuration line. We expect hardware noise to affect results, and testing the Madelon noise-robustness finding on a real QPU is the single most important validation this work requires.

Quantum Machine Learning will not replace classical approaches tomorrow. But the physics is real, the hardware is improving, and the mathematical framework is solid. The noise-robustness result we found is a small but genuine piece of evidence that NISQ-era quantum devices can already do something useful that classical kernels cannot. That is worth understanding, and worth building on.

REFERENCES

[1] G. E. Moore, “Cramming More Components onto Integrated Circuits,” Electronics, vol. 38, no. 8, pp. 114–117, Apr. 1965.

[2] M. M. Waldrop, “The Chips Are Down for Moore’s Law,” Nature, vol. 530, pp. 144–147, Feb. 2016.

[3] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum Machine Learning,” Nature, vol. 549, pp. 195–202, Sep. 2017.

[4] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information. Cambridge: Cambridge University Press, 2010.

[5] P. Kaye, R. Laflamme, and M. Mosca, An Introduction to Quantum Computing. Oxford: Oxford University Press, 2007.

[6] A. Einstein, B. Podolsky, and N. Rosen, “Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?” Phys. Rev., vol. 47, pp. 777–780, May 1935.

[7] W. H. Zurek, “Decoherence and the Transition from Quantum to Classical,” Phys. Today, vol. 44, no. 10, pp. 36–44, Oct. 1991.

[8] J. Preskill, “Quantum Computing in the NISQ Era and Beyond,” Quantum, vol. 2, p. 79, Aug. 2018.

[9] C. Cortes and V. Vapnik, “Support-Vector Networks,” Mach. Learn., vol. 20, pp. 273–297, Sep. 1995.

[10] M. Schuld, I. Sinayskiy, and F. Petruccione, “Circuit-Centric Quantum Classifiers,” Phys. Rev. A, vol. 101, p. 032308, Mar. 2020.

[11] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, “Barren Plateaus in Quantum Neural Network Training Landscapes,” Nat. Commun., vol. 9, p. 4812, Nov. 2018.

[12] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre, “Data Re-uploading for a Universal Quantum Classifier,” Quantum, vol. 4, p. 226, Feb. 2020.

[13] V. Havlíček et al., “Supervised Learning with Quantum-Enhanced Feature Spaces,” Nature, vol. 567, pp. 209–212, Mar. 2019.

[14] V. Bergholm et al., “PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations,” arXiv:1811.04968 [quant-ph], 2022.

[15] G. Aleksandrowicz et al., “Qiskit: An Open-source Framework for Quantum Computing,” Zenodo, 2019. doi: 10.5281/zenodo.2562110.

[16] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

APPENDICES

APPENDIX A: ENVIRONMENT SETUP AND INSTALLATION COMMANDS

For future students at Erzurum Technical University who wish to reproduce or extend these experiments, the core installation commands for the two quantum computing frameworks used in this thesis are listed below (see also Figure 5.7 in Chapter 5 for the corresponding Qiskit installation screenshot).

- pip install pennylane
- pip install qiskit
- pip install qiskit_machine_learning
- pip install qiskit_algorithms

pennylane installs the main experimental framework used throughout this thesis. qiskit installs the core Qiskit framework; qiskit_machine_learning adds VQC, QSVM, and QuantumKernel; qiskit_algorithms adds classical optimizers including COBYLA and L-BFGS-B. All packages are required to reproduce the QSVM experiments.

STANDARDS AND CONSTRAINTS FORM

Regarding the standards and constraints followed in the preparation of the project, please answer the following questions.

1. What is the design dimension of your project? (Is it a new project? A repetition of an existing project? Part of a larger project? What percentage of the total project does your design represent?)

This project builds on an existing area of research (Quantum Machine Learning) but constitutes a newly developed, original implementation rather than a repetition of a prior project. The comparative benchmarking framework — covering VQC, QSVM, and Classical SVM across four datasets, together with the Data Re-uploading strategy and the Barren Plateau experiments was designed and built entirely by us for this senior project. It is not part of a larger ongoing project; it represents 100% of the design and implementation work submitted here.

2. Did you formulate and solve an engineering problem yourselves in your project? Explain.

Yes. We formulated the following problem ourselves: under NISQ-era hardware constraints (shallow circuits, limited qubit counts, no error correction), does Quantum Machine Learning offer any measurable advantage over a well-established classical method? We solved this by designing a controlled experimental setup — consistent preprocessing, matched train/test splits, and comparable evaluation metrics across all three models — and by identifying and diagnosing two concrete engineering problems along the way: the Barren Plateau degradation in VQC as qubit count increases, and the accuracy gap between Angle and Amplitude encoding, for which we implemented a Data Re-uploading mitigation.

3. Which knowledge and skills acquired in previous courses did you use?

We drew directly on three courses: Linear Algebra (vector spaces, inner products, and matrix decompositions — the same mathematical language used for quantum state vectors, unitary gate operations, and the kernel/feature-space formulations behind both Classical SVM and QSVM), Data Science (preprocessing, train/test splitting, cross-validation, and performance evaluation methodology applied consistently across all three models), and Data Mining for Cybersecurity (working with high-dimensional, noisy, and adversarial-style data — directly applicable to the Madelon dataset, which we specifically chose to stress-test noise robustness).

4. What engineering standards did you use or take into consideration? (List the codes and names of the standards used and/or required, relevant to your project topic.)

As this is an academic research study rather than a commercial engineering deliverable, no formal industry hardware/product standard applies. Instead, we followed established academic and scientific standards: the IEEE Code of Ethics declared in this document; IEEE citation format for all references; open-source licensing terms of the frameworks used (PennyLane,

Qiskit, scikit-learn); and standard machine learning research practices for reproducibility — fixed random seeds, documented train/test splits, and clearly reported evaluation metrics — so that our results can be independently verified and reproduced.
5. What realistic constraints did you use or take into consideration? Please fill in the blanks with appropriate answers.
a) Economy
As an academic study, the project was designed to require zero monetary cost — relying entirely on free, open-source frameworks (PennyLane, Qiskit, scikit-learn) and standard university/personal laptop hardware, with no paid cloud quantum hardware access.
b) Environmental issues
All experiments ran on classical simulators on standard laptop hardware rather than energy-intensive cloud quantum processors, keeping the study's energy footprint comparable to routine coursework computing.
c) Sustainability
The full codebase, datasets, and documentation are open and self-contained, allowing future ETÜ students to reproduce, extend, or build on this work as a reference baseline without any licensing or access barriers.
d) Manufacturability
Not applicable — this is a computational/algorithmic research study, not a physical product.
e) Ethics
The study adheres to the IEEE Code of Ethics declared in this document and to standard academic research integrity practices: all datasets (Iris, Wine, Breast Cancer, Madelon) are public benchmark datasets with no personal, sensitive, or human-subject data, and all results are reported honestly, including negative/underwhelming findings (e.g., Classical SVM's below-random performance on Madelon).
f) Health
Not applicable — no human subjects, clinical data, or health outcomes were involved.
g) Safety
Not applicable no physical systems, hardware experiments, or safety-critical components were involved; all work was purely computational.
h) Social and political issues

As an academic contribution, this study's main social value is educational: it provides a documented, reproducible baseline for future ETÜ Computer Engineering students studying Quantum Machine Learning. Broader societal implications of quantum computing (e.g., future impact on cryptography) were considered out of scope for this study.